ELEC 374 Digital Systems II

Machine Problem 1

Owen Swaine – 20386155

March 11, 2026

Table of Contents

Part 1: ***1***
CUDA Device Information 1
Output of code 1
Code for Part 1 2
Part 2: ***4***
a) Host–Device Data Transfer Time 4
b) CPU vs GPU Matrix Multiplication Time 5
c) Matrix Multiplication Performance Analysis 5
c.a) How many times is each element of each input matrix loaded? 6
c.b) What is the cGMA ratio? 7
Output of Code 8
Code For Part 2 9
Statement of Originality ***15***
References ***15***

Part 1:

CUDA Device Information

The CUDA device query program (Figure 1) reports that the system contains 1 CUDA-capable GPU, identified as NVIDIA GeForce RTX 3060 Ti. This GPU operates with a clock rate of 1,665,000 kHz (1.665 GHz) and contains 38 streaming multiprocessors (SMs). Each SM contains multiple CUDA cores, resulting in a total of 4,864 CUDA cores (see note about this), which are responsible for executing parallel threads. The device uses a warp size of 32 threads, meaning that threads are executed in groups of 32 simultaneously on the GPU.

The GPU has 8,589,410,304 bytes (approximately 8 GB) of global memory, which is the main memory accessible by all threads on the GPU. In addition, it includes 65,536 bytes of constant memory, which is a small read-only memory space optimized for broadcasting values to many threads.

Each thread block can access 49,152 bytes of shared memory, which is a fast on-chip memory shared among threads within the same block. Shared memory is often used to optimize performance by reducing accesses to slower global memory. The device also provides 65,536 registers per block, which are used to store variables for threads during execution. The GPU supports a maximum of 1,024 threads per block, allowing large numbers of threads to run concurrently. The maximum block dimensions are 1024 × 1024 × 64, and the maximum grid dimensions are 2,147,483,647 × 65,535 × 65,535, which define how blocks can be organized when launching kernels.

Output of code

```
Number of CUDA devices: 1

Device 0: NVIDIA GeForce RTX 3060 Ti
Clock Rate: 1665000
Streaming Multiprocessors: 38
CUDA Cores: 4864
Warp Size: 32
Global Memory: 8589410304 bytes
Constant Memory: 65536 bytes
Shared Memory per Block: 49152 bytes
Registers per Block: 65536
Max Threads per Block: 1024
Max Block Dimensions: 1024 x 1024 x 64
Max Grid Dimensions: 2147483647 x 65535 x 65535
```

Figure 1: Output of the CUDA device information script (part 1 of the assignment)

Code for Part 1

Note: the code is inspired by the following source from class [1].
Also, I got the number of CUDA cores per Streaming Multiprocessor, by using the fact that it depends on the GPU. NVIDIA defines different SM designs for each compute capability version. For example, Ampere GPUs (compute capability 8.x) contain up to 128 FP32 cores per SM, while earlier architectures such as Pascal and Volta contain 64 cores per SM. The architecture versions (Kepler, Maxwell, Pascal, Volta, Ampere, etc.) correspond to the major compute capability numbers reported by the GPU. Therefore, the code gets the architecture using the major number and returns the corresponding number of cores per SM for that architecture.

Table 2: GPU Architecture and Major Revision Numbers

Major Revision Number	NVIDIA GPU Architecture
9	NVIDIA Hopper GPU Architecture
8	NVIDIA Ampere GPU Architecture
7	NVIDIA Volta GPU Architecture
6	NVIDIA Pascal GPU Architecture
5	NVIDIA Maxwell GPU Architecture
3	NVIDIA Kepler GPU Architecture

20.7.1. Architecture

A Streaming Multiprocessor (SM) consists of:

- 64 FP32 cores for single-precision arithmetic operations in devices of compute capability 8.0 and 128 FP32 cores in devices of compute capability 8.6, 8.7 and 8.9,
- 32 FP64 cores for double-precision arithmetic operations in devices of compute capability 8.0 and 2 FP64 cores in devices of compute capability 8.6, 8.7 and 8.9
- 64 INT32 cores for integer math,
- 4 mixed-precision Third-Generation Tensor Cores supporting half-precision (fp16), `__nv_bfloat16`, `tf32`, sub-byte and double precision (fp64) matrix arithmetic for compute capabilities 8.0, 8.6 and 8.7 (see Warp Matrix Functions for details),
- 4 mixed-precision Fourth-Generation Tensor Cores supporting `fp8`, `fp16`, `__nv_bfloat16`, `tf32`, sub-byte and `fp64` for compute capability 8.9 (see Warp Matrix Functions for details),
- 16 special function units for single-precision floating-point transcendental functions,
- 4 warp schedulers.

This information and photos come from NVIDIA [2]. (I had to do it this way because I couldn't find another solution to getting the # of cores)

```
// Owen Swaine - 20386155

#include <stdio.h>
#include <cuda_runtime.h>

// simple helper to estimate CUDA cores per SM based on architecture
//. FROM NVIDIA
int getCoresPerSM(int major)
{
   if (major == 2) return 32;   // Fermi
   if (major == 3) return 192;  // Kepler
   if (major == 5) return 128;  // Maxwell
   if (major == 6) return 64;   // Pascal
   if (major == 7) return 64;   // Volta
   if (major == 8) return 128;  // Ampere
   return 0;                    // unknown archdd
}

int main() {

   int deviceCount;

   // get how many CUDA GPUs are available
   cudaGetDeviceCount(&deviceCount);
```

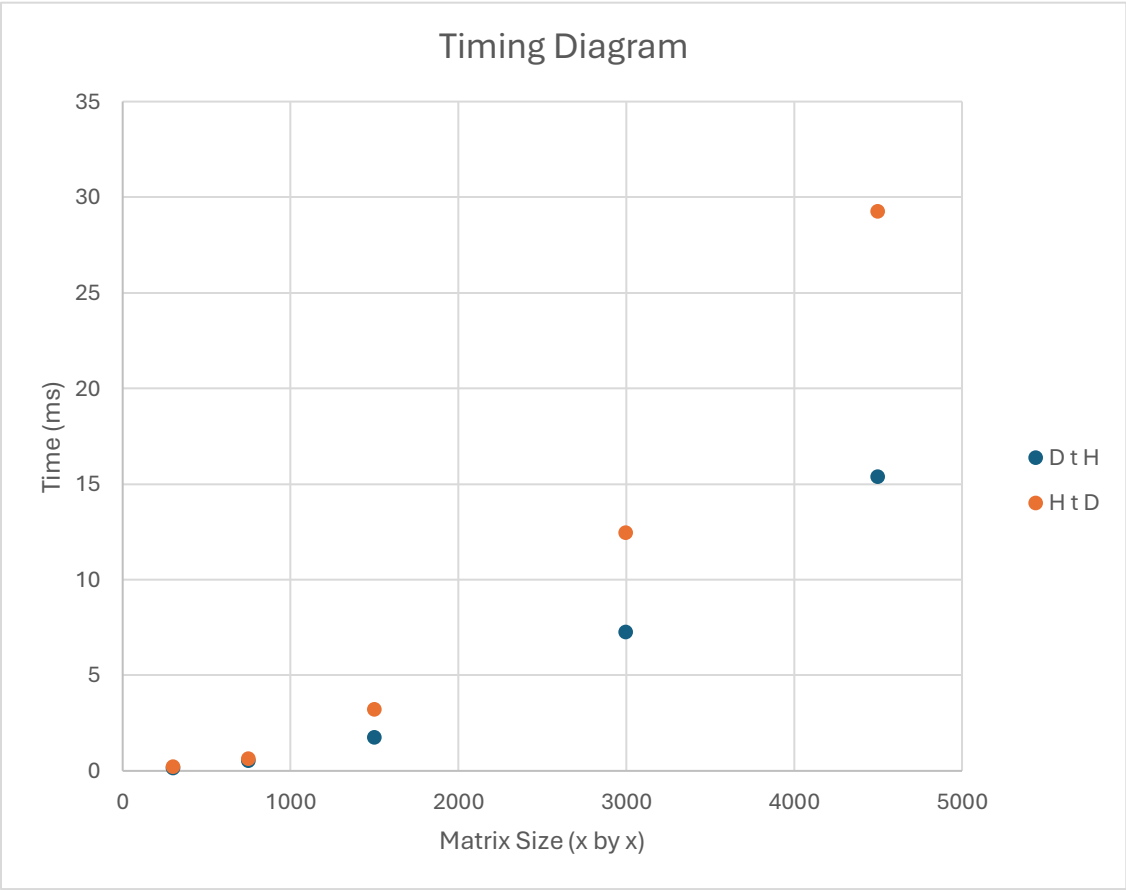
```
  printf("Number of CUDA devices: %d\n\n", deviceCount);

  // loop through each detected GPU (only 1 in the bain lab I had)
  for(int i = 0; i < deviceCount; i++) {

    cudaDeviceProp prop;

    // fill struct with device info
    cudaGetDeviceProperties(&prop, i);

    // total CUDA cores
    int coresPerSM = getCoresPerSM(prop.major);
    int totalCores = coresPerSM * prop.multiProcessorCount;

    printf("Device %d: %s\n", i, prop.name);
    printf("Clock Rate: %d\n", prop.clockRate);
    printf("Streaming Multiprocessors: %d\n", prop.multiProcessorCount);
    printf("CUDA Cores: %d\n", totalCores);
    printf("Warp Size: %d\n", prop.warpSize);
    printf("Global Memory: %zu bytes\n", prop.totalGlobalMem);
    printf("Constant Memory: %zu bytes\n", prop.totalConstMem);
    printf("Shared Memory per Block: %zu bytes\n", prop.sharedMemPerBlock);
    printf("Registers per Block: %d\n", prop.regsPerBlock);
    printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);

    // maximum size of a thread block
    printf("Max Block Dimensions: %d x %d x %d\n",
      prop.maxThreadsDim[0],
      prop.maxThreadsDim[1],
      prop.maxThreadsDim[2]);

    // maximum size of the grid
    printf("Max Grid Dimensions: %d x %d x %d\n\n",
      prop.maxGridSize[0],
      prop.maxGridSize[1],
      prop.maxGridSize[2]);
  }

  return 0;
}
```

Part 2:

a) Host–Device Data Transfer Time

Experiments were performed with matrix sizes 300×300, 750×750, 1500×1500, 3000×3000, and 4500×4500 to measure the time reequired to transfer data between the host (CPU) and device (GPU). The transfer time for the two input matrices M and N from host to device (H→D) and the result matrix P from device to host (D→H) were recorded.

Matrix Size	Host → Device (M,N) Time (ms)	Device → Host (P) Time (ms)
300 × 300	0.200032	0.099840
750 × 750	0.605760	0.499744
1500 × 1500	3.186176	1.710464
3000 × 3000	12.425024	7.232768
4500 × 4500	29.229601	15.349792



The transfer time increases as matrix size increases because more data must be copied between host and device memory. **Question answered:** Host-to-device transfers are consistently slower because two matrices are transferred to the GPU, while only one matrix is transferred back to the host.

b) CPU vs GPU Matrix Multiplication Time

The matrix multiplication time was measured for matrix sizes 300×300 and 750×750 on both the CPU and GPU. For the GPU, two configurations were tested: Single block with one thread, and Blocks with a block width of 1. Only the matrix multiplication computation time was measured, excluding memory allocation and data transfer time.

Matrix Size	CPU Time (ms)	GPU (1 block, 1 thread) (ms)	GPU (block width 1) (ms)
300 × 300	71	14639.24	25.72
750 × 750	1111	242989.70	409.74

The results show that using one GPU thread is extremely slow because the GPU is designed for parallel execution. Running the kernel with only a single thread removes the parallel advantage of the GPU and results in worse performance than the CPU. When using block width 1, many blocks are launched with one thread each. This allows the GPU to execute many threads in parallel across its streaming multiprocessors, resulting in significantly faster performance than the CPU.

If data transfer time is included, the total GPU offload time increases because matrices must be copied between host and device memory. For small matrices, this transfer overhead can represent a noticeable portion of the total runtime. **Question answered:** Therefore, offloading computation to the GPU is not always beneficial for small problem sizes. **Question answered:** GPU offloading becomes beneficial when the problem size is large enough that the parallel computation speedup outweighs the cost of transferring data between the CPU and GPU.

c) Matrix Multiplication Performance Analysis

Findings on Execution time decreases significantly as block width increases from 2 to around 10 to 15 because more threads run in parallel and the GPU is utilized more efficiently. After this point the improvement levels off since the GPU cores are already well utilized. Overall larger matrices benefit the most from parallel execution, and block widths around 10 t 15 give the best performance.

Table 1: Results

Block Width	300×300 (ms)	750×750 (ms)	1500×1500 (ms)	3000×3000 (ms)	4500×4500 (ms)
2	7.07	105	1271	8575	29575
5	1.13	18	177	1351	4743
10	0.456	6.7	58.6	456	1542
15	0.423	5.8	50.2	406	1384
25	0.394	5.67	55.3	448	1507

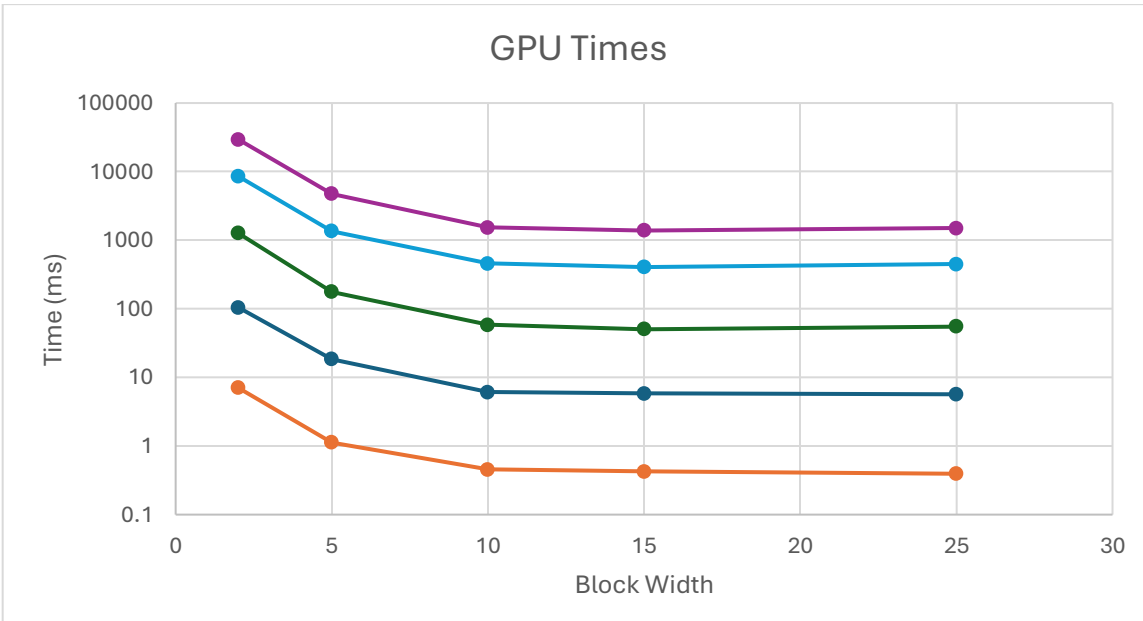


Figure 2: Results Graph. NOTE: The lines are in order bottom to top relative to left to right in the table above.

c.a) How many times is each element of each input matrix loaded?

Each elements of M is loaded WIDTH times, and Each element of N is loaded WIDTH times:
Each thread computes one output element $P_{Row,Col}$, and for that one output it loops over all k values:

$$P_{Row,Col} = \sum_{k=0}^{Width-1} M_{Row,k} \cdot N_{k,Col}$$

So, a fixed element $M_{i,k}$is used in every output in row i, which is WIDTH outputs and a fixed element $N_{k,j}$is used in every output in column j, which is also WIDTH outputs. This is also stated directly in the lecture slides: each row of $M$is loaded WIDTH times and each column of N is loaded WIDTH times [3].

c.b) What is the cGMA ratio?

For one thread computing one output element: the loop runs WIDTH times, each iteration does: 1 multiply and 1 add. So total floating point operations per thread: [3]

$$2 \cdot WIDTH$$

Global memory loads per thread: one load from Meach iteration, and one load from Neach iteration. So total global memory loads per thread: [3]

$$2 \cdot WIDTH$$

Therefore the cGMA ratio is:

$$\frac{2 \cdot WIDTH}{2 \cdot WIDTH} = 1$$

$$\boxed{\text{cGMA} = 1}$$

So this kernel performs 1 floating point operation per global memory load on average, ignoring the final store as instructed. The kernel code clearly shows one load from $M$and one load from $N$for every multiply-add step in the loop

The computed cGMA ratio of 1 indicates that the naive matrix multiplication kernel is memory bound and performs minimal computation per global memory access. This motivates the use of shared memory tiling techniques to increase arithmetic intensity and improve GPU efficiency.

Output of Code

```
==============================
Matrix size: 300 x 300
==============================
H->D time for M and N: 0.200032 ms
D->H time for P     : 0.099840 ms
CPU time            : 71.000000 ms
GPU time 1 block 1 thread: 14639.235352 ms
Test PASSED
GPU time block width 1   : 25.715712 ms
Test PASSED
Approx total GPU offload time: 26.015583 ms
GPU time block width  2: 7.071744 ms   [PASSED]
GPU time block width  5: 1.131520 ms   [PASSED]
GPU time block width 10: 0.455680 ms   [PASSED]
GPU time block width 15: 0.422912 ms   [PASSED]
GPU time block width 25: 0.394240 ms   [PASSED]

==============================
Matrix size: 750 x 750
==============================
H->D time for M and N: 0.605760 ms
D->H time for P     : 0.499744 ms
CPU time            : 1111.000000 ms
GPU time 1 block 1 thread: 242989.703125 ms
Test PASSED
GPU time block width 1   : 409.743347 ms
Test PASSED
Approx total GPU offload time: 410.848877 ms
GPU time block width  2: 105.144318 ms   [PASSED]
GPU time block width  5: 18.365215 ms   [PASSED]
GPU time block width 10: 6.066176 ms   [PASSED]
GPU time block width 15: 5.826496 ms   [PASSED]
GPU time block width 25: 5.675008 ms   [PASSED]

==============================
Matrix size: 1500 x 1500
==============================
H->D time for M and N: 3.186176 ms
D->H time for P     : 1.710464 ms
CPU time            : 9024.000000 ms
GPU time block width  2: 1271.743530 ms   [PASSED]
GPU time block width  5: 177.358841 ms   [PASSED]
GPU time block width 10: 58.621952 ms   [PASSED]
GPU time block width 15: 50.289665 ms   [PASSED]
GPU time block width 25: 55.389183 ms   [PASSED]

==============================
Matrix size: 3000 x 3000
==============================
H->D time for M and N: 12.425024 ms
D->H time for P     : 7.232768 ms
CPU time            : 97404.000000 ms
GPU time block width  2: 8574.946289 ms   [PASSED]
GPU time block width  5: 1351.056396 ms   [PASSED]
GPU time block width 10: 456.099854 ms   [PASSED]
GPU time block width 15: 406.733826 ms   [PASSED]
GPU time block width 25: 448.358398 ms   [PASSED]

==============================
Matrix size: 4500 x 4500
==============================
H->D time for M and N: 29.229601 ms
D->H time for P     : 15.349792 ms
CPU time            : 398692.000000 ms
GPU time block width  2: 29575.480469 ms   [PASSED]
GPU time block width  5: 4743.846680 ms   [PASSED]
GPU time block width 10: 1542.204468 ms   [PASSED]
GPU time block width 15: 1384.105957 ms   [PASSED]
GPU time block width 25: 1506.831421 ms   [PASSED]
```

Code For Part 2

Note the code is inspired by the following sources from class and NVIDIA [1] [3] [2].

```
// Owen Swaine - 20386155.
#include "cuda_runtime.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

#define TOL 1e-3f   // tolerance for CPU vs GPU result check

// RELATES TO PART B
// single-thread GPU version used for comparison
__global__ void matMulSingleThread(float* P, float* M, float* N, int width)
{
   if (blockIdx.x == 0 && threadIdx.x == 0)
   {
      for (int row = 0; row < width; row++)
      {
         for (int col = 0; col < width; col++)
         {
            float sum = 0.0f;
            for (int k = 0; k < width; k++)
            {
               sum += M[row * width + k] * N[k * width + col];
            }
            P[row * width + col] = sum;
         }
      }
   }
}
// RELATES TO PART B AND PART C
// parallel kernel where each thread computes one output element
__global__ void matMulKernel(float* P, float* M, float* N, int width)
{
   int col = blockIdx.x * blockDim.x + threadIdx.x;
   int row = blockIdx.y * blockDim.y + threadIdx.y;

   // make sure thread is inside matrix bounds
   if (row < width && col < width)
   {
      float sum = 0.0f;
      for (int k = 0; k < width; k++)
      {
         sum += M[row * width + k] * N[k * width + col];
      }
      P[row * width + col] = sum;
   }
}
```

```
// RELATES TO PART B AND PART C
// fills input matrices with random values
void fillRandom(float* A, int width)
{
   for (int i = 0; i < width * width; i++)
   {
      A[i] = (float)rand() / RAND_MAX;
   }
}

// RELATES TO PART B AND PART C
// CPU version used for timing and correctness check
void matMulCPU(float* P, float* M, float* N, int width)
{
   for (int row = 0; row < width; row++)
   {
      for (int col = 0; col < width; col++)
      {
         float sum = 0.0f;
         for (int k = 0; k < width; k++)
         {
            sum += M[row * width + k] * N[k * width + col];
         }
         P[row * width + col] = sum;
      }
   }
}

// RELATES TO PART B AND PART C
// checks if GPU output matches CPU output
int compareResults(float* cpu, float* gpu, int width)
{
   for (int i = 0; i < width * width; i++)
   {
      if (fabs(cpu[i] - gpu[i]) > TOL)
      {
         printf("Mismatch at index %d CPU=%f GPU=%f\n", i, cpu[i], gpu[i]);
         return 0;
      }
   }
   return 1;
}

// RELATES TO PART A
// measures host to device transfer time for M and N
float getHtoDTime(float* d_M, float* d_N, float* h_M, float* h_N, int bytes)
{
   cudaEvent_t start, stop;
   float time_ms;
```

```
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start, 0);
    cudaMemcpy(d_M, h_M, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_N, h_N, bytes, cudaMemcpyHostToDevice);
    cudaEventRecord(stop, 0);

    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&time_ms, start, stop);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return time_ms;
}

// RELATES TO PART A
// measures device to host transfer time for P
float getDtoHTime(float* h_P, float* d_P, int bytes)
{
    cudaEvent_t start, stop;
    float time_ms;

    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start, 0);
    cudaMemcpy(h_P, d_P, bytes, cudaMemcpyDeviceToHost);
    cudaEventRecord(stop, 0);

    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&time_ms, start, stop);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return time_ms;
}

// RELATES TO PART B
// times the 1 block 1 thread GPU case
float getKernelTimeSingleThread(float* d_P, float* d_M, float* d_N, int width)
{
    cudaEvent_t start, stop;
    float time_ms;

    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start, 0);
```

```
    matMulSingleThread<<<1, 1>>>(d_P, d_M, d_N, width);
    cudaEventRecord(stop, 0);

    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&time_ms, start, stop);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return time_ms;
}

// RELATES TO PART B AND PART C
// times the parallel kernel for a chosen block width
float getKernelTime2D(float* d_P, float* d_M, float* d_N, int width, int blockWidth)
{
    cudaEvent_t start, stop;
    float time_ms;

    dim3 threads(blockWidth, blockWidth);
    dim3 blocks((width + blockWidth - 1) / blockWidth,
            (width + blockWidth - 1) / blockWidth);

    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start, 0);
    matMulKernel<<<blocks, threads>>>(d_P, d_M, d_N, width);
    cudaEventRecord(stop, 0);

    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&time_ms, start, stop);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    return time_ms;
}

// RELATES TO PART B
// measures CPU matrix multiplication time
double getCPUTime(float* h_P, float* h_M, float* h_N, int width)
{
    clock_t start, stop;
    start = clock();
    matMulCPU(h_P, h_M, h_N, width);
    stop = clock();

    return 1000.0 * (double)(stop - start) / CLOCKS_PER_SEC;
}
```

```
// RELATES TO PART A, PART B, AND PART C
// runs one full test case for one matrix size
void runCase(int width)
{
    int elements = width * width;
    int bytes = elements * sizeof(float);

    float* h_M;
    float* h_N;
    float* h_P_cpu;
    float* h_P_gpu;

    float* d_M;
    float* d_N;
    float* d_P;

    // allocate host memory
    h_M = (float*)malloc(bytes);
    h_N = (float*)malloc(bytes);
    h_P_cpu = (float*)malloc(bytes);
    h_P_gpu = (float*)malloc(bytes);
    // fill input matrices with random values
    fillRandom(h_M, width);
    fillRandom(h_N, width);

    // allocate device memory
    cudaMalloc((void**)&d_M, bytes);
    cudaMalloc((void**)&d_N, bytes);
    cudaMalloc((void**)&d_P, bytes);

    printf("\n==============================\n");
    printf("Matrix size: %d x %d\n", width, width);
    printf("==============================\n");

    // PART A: host to device transfer time
    float h2d = getHtoDTime(d_M, d_N, h_M, h_N, bytes);
    printf("H->D time for M and N: %f ms\n", h2d);

    // PART A: device to host transfer time
    cudaMemset(d_P, 0, bytes);
    float d2h = getDtoHTime(h_P_gpu, d_P, bytes);
    printf("D->H time for P     : %f ms\n", d2h);

    // PART B: CPU timing
    double cpu_time = getCPUTime(h_P_cpu, h_M, h_N, width);
    printf("CPU time            : %f ms\n", cpu_time);

    // copy actual input matrices to device before kernel tests
    cudaMemcpy(d_M, h_M, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_N, h_N, bytes, cudaMemcpyHostToDevice);
```

```
// PART B: only needed for 300x300 and 750x750
if (width == 300 || width == 750)
{
   // 1 block 1 thread case
   cudaMemset(d_P, 0, bytes);
   float gpu1 = getKernelTimeSingleThread(d_P, d_M, d_N, width);
   cudaMemcpy(h_P_gpu, d_P, bytes, cudaMemcpyDeviceToHost);

   printf("GPU time 1 block 1 thread: %f ms\n", gpu1);
   if (compareResults(h_P_cpu, h_P_gpu, width))
      printf("Test PASSED\n");
   else
      printf("Test FAILED\n");

   // block width 1 case
   cudaMemset(d_P, 0, bytes);
   float gpu_bw1 = getKernelTime2D(d_P, d_M, d_N, width, 1);
   cudaMemcpy(h_P_gpu, d_P, bytes, cudaMemcpyDeviceToHost);

   printf("GPU time block width 1   : %f ms\n", gpu_bw1);
   if (compareResults(h_P_cpu, h_P_gpu, width))
      printf("Test PASSED\n");
   else
      printf("Test FAILED\n");

   // approximate full offload time
   printf("Approx total GPU offload time: %f ms\n", h2d + gpu_bw1 + d2h);
}

// PART C: block width experiments
int blockWidths[5] = {2, 5, 10, 15, 25};

for (int i = 0; i < 5; i++)
{
   int bw = blockWidths[i];

   cudaMemset(d_P, 0, bytes);
   float gpu_time = getKernelTime2D(d_P, d_M, d_N, width, bw);
   cudaMemcpy(h_P_gpu, d_P, bytes, cudaMemcpyDeviceToHost);

   printf("GPU time block width %2d: %f ms   ", bw, gpu_time);

   if (compareResults(h_P_cpu, h_P_gpu, width))
      printf("[PASSED]\n");
   else
      printf("[FAILED]\n");
}

// free host memory
free(h_M);
free(h_N);
```

```
    free(h_P_cpu);
    free(h_P_gpu);

    // free device memory
    cudaFree(d_M);
    cudaFree(d_N);
    cudaFree(d_P);
}

int main()
{
    srand((unsigned int)time(NULL));   // seed random numbers

    int sizes[5] = {300, 750, 1500, 3000, 4500};

    // run all required matrix sizes
    for (int i = 0; i < 5; i++) [1]
    {
        runCase(sizes[i]);
    }

    cudaDeviceReset();
    return 0;
}
```

Statement of Originality

"I do hereby verify that this machine problem submission is my own work and contains my own original ideas, concepts, and designs. No portion of this report or code has been copied in whole or in part from another source, with the possible exception of properly referenced material."

References

[1] A. Afsahi, E. Shama and A. Sojoodi, "ELEC 374 – GPU Servers and CUDA Environment Tutorial," Queen’s University, Smith Engineering, Kingston, 2026.

[2] NVIDIA Corporation, "CUDA C Programming Guide," NVIDIA Corporation, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/. [Accessed 12 March 2026].

[3] A. Afsahi, "GPU Architectures and Computing," Queen's University, Kingston, Ontario, 2026.